

Phone-to-3D Project: Technical Report

Nil Sila Ulucan, Tibet Gül

1 Introduction and Project Goal

Phone-to-3D is an open-ended computer vision pipeline that automatically reconstructs a 3D point cloud and mesh model from a sequence of photos captured with a phone camera. The core goal of the project is to obtain a 3D digital twin of a real-world scene or object using only ordinary phone-camera imagery, without any additional hardware such as LiDAR or depth sensors, by applying Structure-from-Motion (SfM) and Multi-View Stereo (MVS) techniques. The pipeline is implemented as a command-line Python application. Its primary and currently validated workflow is photo-to-3D: it takes a folder of still images (e.g. the Gerard Hall benchmark dataset) and reconstructs a 3D model directly from them. As an additional input mode, the codebase also supports extracting frames from a video file via FFmpeg and feeding those frames into the same pipeline, but this path has not yet been validated end-to-end.

2 System Architecture and Pipeline Stages

The project is built around a four-stage end-to-end flow defined in `src/pipeline/full_pipeline.py`:

2.1 1. Input Handling (Direct Image Set or Video to Frames)

In the primary, validated workflow the input is already a directory of photos (e.g. a static benchmark dataset such as Gerard Hall), so this step is skipped entirely and the images are passed directly to the SfM stage. As an optional secondary path, the `video_to_frames.py` module can use FFmpeg to extract JPEG frames from an input video at a rate of 10 frames per second (`FRAME_RATE`) and feed those frames into the same downstream pipeline.

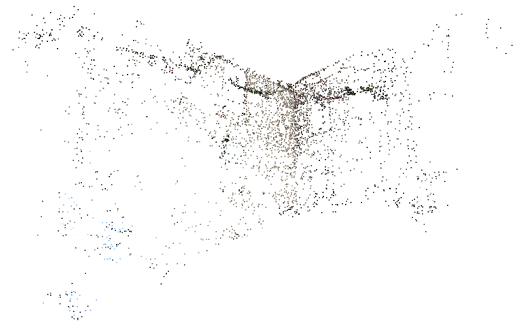


Figure 1: Sparse point cloud from the COLMAP SfM stage on a subset of the Gerard Hall photo set, visualized with Open3D.

2.2 2. Sparse Reconstruction with COLMAP (SfM)

COLMAP is run through three steps: *feature_extractor* (SIFT feature extraction with a limit of 8192 features), *exhaustive_matcher* (feature matching across all image pairs), and *mapper* (incremental camera pose estimation and sparse 3D point cloud generation), followed by *model_converter* to export a human-readable TXT model. The mapper’s inlier and match-count thresholds were lowered so reconstruction does not fail on small or weakly matched image sets.

2.3 3. Dense Reconstruction and Mesh Generation with OpenMVS

The OpenMVS tools (*InterfaceCOLMAP*, *DensifyPointCloud*, *ReconstructMesh*, *RefineMesh*, *TextureMesh*) have been tested manually on Windows – the `DensifyPointCloud-*.log` files in the repository confirm this. However, `run_openmvs.py` does not yet call these tools automatically: it currently only runs an emergency bypass that converts COLMAP’s sparse `points3D.bin` to `.ply` via *model_converter*, giving a viewable sparse cloud but no dense mesh. Automating this chain is the project’s main next step.

2.4 4. Visualization

`src/viewer/viewer.py` loads the resulting `.ply` file with the Open3D library and displays it in an interactive 3D window. This step is optionally triggered from `main.py` via the `-view` flag.

3 Project Structure: Key Files

- `main.py`: argparse-based CLI entry point with `-name`, `-video` and `-view` options. Also contains a fixed development block that runs the pipeline directly on the Gerard Hall photo dataset for automated testing.
- `src/config.py`: centralizes platform-specific (Windows/Unix) paths to the FFmpeg, COLMAP and OpenMVS binaries and the global `FRAME_RATE` setting, so the same codebase runs on a Windows COLMAP install or on macOS/Linux with system-wide tools.
- `src/pipeline/full_pipeline.py`: orchestrates the four stages (input handling, COLMAP, OpenMVS, output) and returns the path to the final mesh `.ply`.
- `src/pipeline/video_to_frames.py`: optional, secondary path that calls FFmpeg to extract frames from a video file.
- `src/pipeline/run_colmap.py`: runs COLMAP's *feature_extractor*, *exhaustive_matcher* and *mapper* with tuned thresholds, then exports the sparse model to TXT.
- `src/pipeline/run_openmvs.py`: currently only runs the "emergency bypass" (`points3D.bin` → `.ply` via COLMAP); the OpenMVS dense/mesh calls are not yet wired in.
- `src/utils/colmap_validator.py`: sanitizes COLMAP's `images.txt` (filenames only, verified existence, Unix line endings); written for OpenMVS's *InterfaceCOLMAP* step but not yet called from the pipeline.
- `src/utils/file_utils.py` / `logging_utils.py`: small helpers for directory creation and consistent logging across all pipeline modules.
- `src/viewer/viewer.py`: loads and displays the final `.ply` point cloud/mesh with Open3D.
- `data/`: working directory tree containing `input_images/` (photo datasets, e.g. Gerard Hall), `input_video/` (`blake_hall.mp4`, `gerard_hall.mp4`), `frames/` (extracted frames), and `colmap/` (per-run COLMAP/OpenMVS workspaces and outputs).

4 Technologies Used

Python 3 (orchestration/CLI), **FFmpeg** (video frame extraction), **COLMAP** (feature extraction, matching and Structure-from-Motion), **OpenMVS** (dense point cloud and mesh generation), **Open3D** (point cloud visualization), and **NumPy/OpenCV** (supporting image-processing dependencies).

5 Challenges Encountered

A major challenge was that COLMAP's model files could not be processed directly by OpenMVS: on Windows, the path format and line-ending characters in `images.txt` caused *InterfaceCOLMAP* to crash. A fix (`colmap_validator.py`) was written but is not yet integrated into the pipeline. Additionally, COLMAP's mapper failed on small or weakly matched image sets under default thresholds; this was fixed by lowering the relevant inlier and match-count thresholds.

6 Next Steps

The priority goals for the continuation of the project are:

- **Automate the OpenMVS chain**: extend `run_openmvs.py` to call `colmap_validator.py`, then run *InterfaceCOLMAP* → *DensifyPointCloud* → *ReconstructMesh* → *RefineMesh* → *TextureMesh* via subprocess, as `run_colmap.py` does for COLMAP, removing the bypass.
- Validate the full pipeline on real phone-recorded video samples (`blake_hall.mp4`, `gerard_hall.mp4`), and add automated quality metrics (point density, coverage ratio).